

哈爾濱工業大學

《神经计算与类脑智能 原理》课程实验报告

题目 多层感知器（MLP）@ MNIST

专业 人工智能

学号 2023112451

姓名 顾雨航

一. 背景简介/问题描述

MNIST 是机器学习和模式识别领域中最经典的入门数据集之一，主要用于手写数字识别任务。该数据集由 0 到 9 共 10 个类别构成，其中训练集包含 60000 张手写数字图片，测试集包含 10000 张图片。每张图片均为 28×28 像素的灰度图像，数据规模适中、类别清晰、标注规范，因此非常适合作为神经网络分类实验的基础数据集。

本实验的主要任务是结合课程要求，使用 PyTorch 框架搭建一个多层感知器（MLP）模型，对 MNIST 数据集进行分类训练与测试，并分析模型在手写数字识别问题上的性能表现。实验要求不仅包括模型的设计与实现，还包括数据集加载、预处理、训练过程控制、模型测试以及结果分析。

多层感知器属于前馈神经网络的一种，能够通过多层线性变换和非线性激活函数对输入特征进行逐层映射，从而学习输入数据与输出类别之间的对应关系。在本实验中，由于 MNIST 图像尺寸较小、任务难度适中，MLP 既能够满足实验要求，又能够较直观地体现神经网络从输入到输出的映射过程，因此适合作为实验一的主要模型。

本实验最终实现了两层隐藏层的 MLP 模型，并在测试集上取得了 98.38% 的分类准确率。为了进一步分析网络结构对模型性能的影响，又额外设计了一组对比实验，将两层隐藏层改为一层隐藏层，在其他训练条件保持不变的情况下重新训练和测试，用于比较不同网络深度对分类性能的影响。

二. 算法介绍

2.1 多层感知器基本原理

多层感知器（Multilayer Perceptron, MLP）是一种典型的全连接前馈神经网络。它通常由输入层、一个或多个隐藏层以及输出层构成。输入层负责接收原始特征，隐藏层负责对特征进行非线性变换，输出层负责给出最终分类结果。

对一个分类任务而言，MLP 的核心思想可以概括为：

1. 将输入特征映射到高维隐藏空间；
2. 在隐藏空间中利用非线性激活函数提取更有判别力的特征；
3. 通过输出层得到各类别的得分；
4. 使用损失函数衡量预测结果与真实标签之间的误差；
5. 利用反向传播算法更新网络参数，使模型预测结果不断逼近真实标签。

本实验采用的激活函数为 ReLU（Rectified Linear Unit）。ReLU 在深度学习中应用广泛，具有计算简单、梯度传播效果较好等优点，可以有效缓解传统激活函数容易出现的梯度消失问题。

在输出层之后，本实验没有手动加入 Softmax，而是直接使用交叉熵损失函数 CrossEntropyLoss。原因在于 PyTorch 的 CrossEntropyLoss 已经在内部完

成了 softmax 与对数运算的结合处理，因此无需额外重复计算。

2.2 反向传播与参数更新

训练过程中，模型先进行前向传播，得到输入样本在 10 个类别上的输出分数。随后通过交叉熵损失函数计算当前预测结果与真实标签之间的损失值，再对损失函数求梯度并使用反向传播算法将梯度传回网络各层。最后，优化器依据梯度更新各层参数，使损失逐步减小。

本实验使用带动量的随机梯度下降优化器 SGD。动量项的引入能够在一定程度上加快收敛速度，并降低参数更新过程中震荡过大的问题。训练过程中随着 epoch 增加，模型能够逐步学习到更加稳定且有效的数字分类特征。

2.3 本实验采用的模型结构

本实验主模型采用两层隐藏层结构，其网络形式如下：

输入层：784 维

第一隐藏层：Linear(784, 512) + ReLU + Dropout(0.2)

第二隐藏层：Linear(512, 256) + ReLU + Dropout(0.2)

输出层：Linear(256, 10)

其中，输入的 28×28 图像首先被展平成 784 维向量，然后送入全连接网络。为减轻过拟合风险，本实验在隐藏层后加入了 Dropout，丢弃率设置为 0.2。

主模型参数量计算如下：

1. 第一层全连接： $784 \times 512 + 512 = 401920$

2. 第二层全连接： $512 \times 256 + 256 = 131328$

3. 输出层全连接： $256 \times 10 + 10 = 2570$

因此主模型总参数量为： $401920 + 131328 + 2570 = 535818$

三. 算法实现

3.1 实验环境

本实验在 Windows 环境下完成，使用 Anaconda 虚拟环境 nlplab 运行 Python 程序，深度学习框架为 PyTorch。实验过程中实际检测到可用 CUDA 设备，因此训练和测试均可在 GPU 环境下完成。

项目目录按照较规范的工程结构进行组织，主要文件如下：

1. train.py: 训练脚本
2. eval.py: 测试脚本
3. src/mlp.py: MLP 模型结构定义

4. src/dataset.py: 数据集加载与 DataLoader 构建
5. src/config.py: 模型与训练超参数配置
6. ckpt/: 保存训练得到的模型参数和训练历史结果

3.2 数据加载与预处理

实验中使用 torchvision.datasets.MNIST 自动下载并读取数据集。为了提高训练效果，图片数据在输入模型前进行了以下预处理：

1. 使用 ToTensor() 将图像从像素矩阵转换为张量；
2. 使用 Normalize((0.1307,), (0.3081,)) 对灰度图像进行标准化；
3. 在训练阶段对训练集启用 shuffle=True，以增强样本随机性；
4. 在测试阶段不打乱测试集顺序，仅用于性能评估。

由于多层感知器只能接收向量输入，因此在模型中通过 Flatten() 将每张 28×28 的灰度图片展平为长度为 784 的一维向量，再输入全连接层。

3.3 训练参数设置

本实验主模型采用如下超参数：

1. 训练轮数 epoch_size: 10
2. 训练批大小 batch_size: 64
3. 测试批大小 test_batch_size: 1000
4. 学习率 lr: 0.01
5. 动量 momentum: 0.9
6. Dropout: 0.2
7. 优化器: SGD
8. 随机种子: 42

这些参数在保证训练稳定性的同时，也能够使模型在较短时间内完成较高精度的训练。

3.4 训练流程

训练脚本的核心流程如下：

1. 读取配置参数并设置随机种子；
2. 加载 MNIST 训练集和测试集；
3. 初始化 MLP 模型；
4. 定义损失函数 CrossEntropyLoss；
5. 定义优化器 SGD；
6. 对训练集进行多轮迭代训练；
7. 每个 epoch 结束后在测试集上进行评估；
8. 保存测试准确率最高时对应的模型参数；
9. 将每轮训练结果记录到 history.json 文件中。

训练时，模型首先进入 train 模式。每个 batch 依次完成前向传播、损失计算、反向传播和参数更新。当一个 epoch 结束后，再切换到 eval 模式，在测试集上计算平均损失和分类准确率。

3.5 训练结果

本实验主模型在 10 个 epoch 上的训练结果如下：

表 1 主模型训练与测试结果

Epoch	Train Loss	Test Loss	Test Accuracy
1	0.321979	0.132910	95.80%
2	0.120906	0.092934	97.10%
3	0.086104	0.073422	97.63%
4	0.065605	0.066334	97.86%
5	0.051965	0.064884	97.91%
6	0.042623	0.060815	98.09%
7	0.034542	0.059574	98.13%
8	0.029626	0.055073	98.27%
9	0.025460	0.056011	98.38%
10	0.021854	0.057885	98.28%

从训练结果可以看出，随着训练轮数增加，训练损失持续下降，模型的拟合能力不断增强；测试准确率在前期增长较快，后期趋于稳定，在第 9 轮达到最高值 98.38%。第 10 轮时训练损失继续下降，但测试准确率略有下降，这说明模型后期开始出现轻微过拟合趋势。因此，本实验将第 9 轮对应的模型参数保存为最佳模型。

3.6 测试结果

在最佳模型参数加载后，使用 eval.py 对测试集进行独立评估，得到结果如下：

- 1. device: cuda
- 2. checkpoint: ckpt/best_mlp_mnist.pt
- 3. test_loss: 0.056011
- 4. test_accuracy: 98.38%

该结果说明，主模型已经能够对绝大多数手写数字进行正确分类，实验目标顺利实现。

四. 讨论及结论

4.1 主实验结果分析

本实验成功完成了基于 PyTorch 的 MLP 手写数字分类任务。实验结果表明，两层隐藏层的多层感知器在 MNIST 数据集上能够取得较高的分类精度，其最佳测试准确率达到 98.38%。这说明即使不使用卷积结构，仅使用全连接网络，在较为标准且难度适中的图像分类任务上，仍然可以取得较好的识别效果。

从损失变化趋势来看，训练损失在整个训练过程中持续下降，说明模型在不断学习训练样本中的有效特征。测试损失整体也呈下降趋势，但在最后一轮略有回升，说明当训练轮数继续增加时，模型的泛化能力提升已经趋于饱和，甚至可能开始出现轻微过拟合。因此，训练过程中保存最佳模型而不是最后一轮模型是合理的。

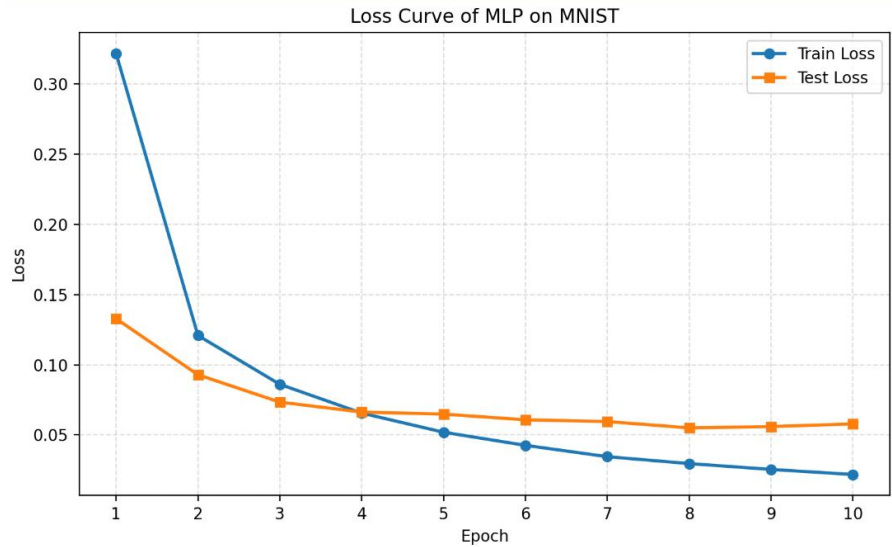


图 1 主模型训练损失与测试损失变化曲线

从准确率变化来看，模型在前几轮训练后便迅速达到较高水平，说明 MNIST 数据集本身结构较清晰，MLP 可以较快学到数字图像的主要判别信息。后续提升幅度逐渐缩小，表明网络已经逐步逼近当前结构下的性能上限。

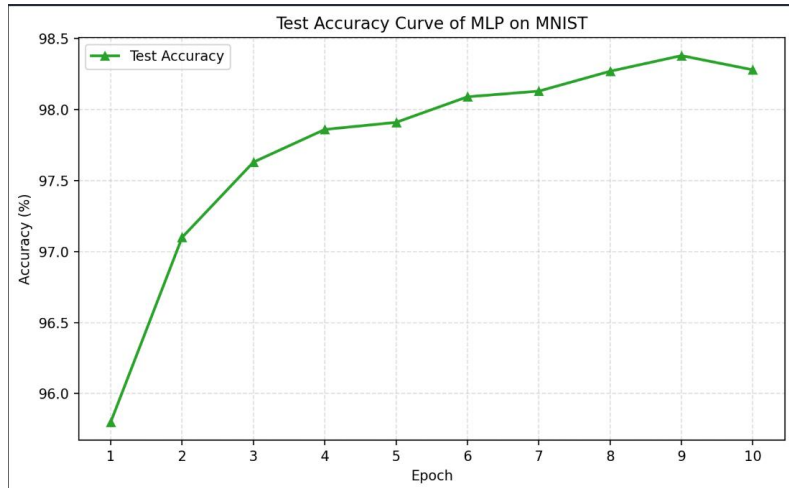


图 2 主模型测试准确率变化曲线

4.2 补充对比实验：减少一层隐藏层

为了进一步分析网络深度对模型性能的影响，本实验额外构建了一个一层隐藏层的对比版本。该模型保持数据集、训练轮数、学习率、优化器、Dropout 等条件不变，仅将主模型中的两层隐藏层结构简化为一层隐藏层。

对比模型结构如下：

输入层：784 维

隐藏层：Linear(784, 512) + ReLU + Dropout(0.2)

输出层: Linear(512, 10)

对比模型参数量计算如下:

- 1. 第一层全连接: $784 \times 512 + 512 = 401920$
- 2. 输出层全连接: $512 \times 10 + 10 = 5130$

因此对比模型总参数量为: $401920 + 5130 = 407050$

与主模型相比, 对比模型减少的参数量为: $535818 - 407050 = 128768$, 约减少 24% 的参数。

一层隐藏层模型在 10 个 epoch 上的结果如下:

表2 一层隐藏层对比模型训练与测试结果

Epoch	Train Loss	Test Loss	Test Accuracy
1	0.267601	0.125309	96.35%
2	0.113054	0.083041	97.50%
3	0.080799	0.073047	97.69%
4	0.060857	0.063981	98.06%
5	0.049337	0.062952	97.93%
6	0.040612	0.057807	98.15%
7	0.033492	0.058001	98.14%
8	0.029279	0.056219	98.22%
9	0.024291	0.057626	98.36%
10	0.022245	0.055977	98.19%

其最佳评估结果为:

- 1. device: cuda
- 2. checkpoint: ckpt/best_mlp_one_hidden_mnist.pt
- 3. test_loss: 0.057626
- 4. test_accuracy: 98.36%

将主模型与对比模型结果进行汇总, 可得:

表3 主模型与对比模型结果比较

模型	Test Loss	Test Accuracy	Parameter Count
主模型 (两层隐藏层)	0.056011	98.38%	535818
对比模型 (一层隐藏层)	0.057626	98.36%	407050

差异分析如下:

- 1. 一层隐藏层模型的测试准确率比主模型低 0.02 个百分点;
- 2. 一层隐藏层模型的测试损失略高于主模型;
- 3. 一层隐藏层模型参数量减少约 24%, 结构更简单;
- 4. 在 MNIST 这样相对简单的分类任务上, 减少一层隐藏层并没有造成明显性

能下降。

这一结果说明，在 MNIST 数据集上，即使模型深度降低，一层隐藏层的 MLP 依然能够达到很高的分类精度。这是因为 MNIST 手写数字图像的任务复杂度相对较低，数字类别边界较清晰，单隐藏层网络已经具备足够的特征表达能力。与此同时，两层隐藏层模型在测试损失和最终准确率上仍略优于一层隐藏层模型，说明增加网络深度仍然可以在一定程度上提升模型的表达能力和泛化表现。

4.3 实验结论

通过本实验可以得到以下结论：

1. 使用 PyTorch 搭建的 MLP 模型能够较好地完成 MNIST 手写数字分类任务；
2. 主模型采用两层隐藏层结构，在测试集上取得了 98.38% 的准确率，说明模型设计合理、训练有效；
3. 训练过程中损失持续下降、精度逐步提升，表明模型能够稳定收敛；
4. 通过补充对比实验发现，在 MNIST 任务上，一层隐藏层模型与两层隐藏层模型的性能差距很小，说明该任务对网络深度的要求并不高；
5. 尽管差距较小，但两层隐藏层模型仍在测试损失和准确率上略优，因此本实验最终将其作为主实验模型。

总体来看，本实验不仅完成了实验要求中的 MLP 模型训练与测试，也通过结构对比分析进一步理解了网络深度对模型性能的影响，达到了较好的实验效果。

参考文献

- [1] LeCun Y, Bottou L, Bengio Y, Haffner P. Gradient-Based Learning Applied to Document Recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [2] PyTorch Official Documentation. <https://pytorch.org/>
- [3] MNIST Handwritten Digit Database. <http://yann.lecun.com/exdb/mnist/>

注：报告中图的下方要有图题（如图 1. XXXX），表格需要用三线表，表头需列在表的上方（如表 1. XXXX）。图表居中排列。